R3.04 – Qualité de développement
Introduction et UML

Pierre Pomeret-Coquot
pierre.pomeret@iut-rodez.fr

Université Toulouse Capitole – IUT de Rodez

2026 – 2027





CC BY

Plan du cours

1. Introduction
   - La ressource R3.04
   - Généralités

2. UML – Diagramme de classes
   - Généralités
   - Les « blocs » (classes, paquetages, etc.)
   - Les relations

CC BY

La ressource R3.04 – Qualité de développement

Ressource R3.04 — **Qualité de développement**

Informatique > Développement > Qualité de développement

Descriptif détaillé

Objectif
L'objectif de cette ressource est d'approfondir la qualité de développement.
Savoirs de référence étudiés
- Approfondissement des concepts de développement orienté objet (par ex. : SOLID, notions de dépendance et de couplage…)
- Compréhension et mise en œuvre de patrons de conception, éléments d'architecture logicielle, restructuration de code (refactoring)
- Rédaction de la documentation du code (par ex. : javadoc, doxygen…)
- Structuration de l'application (modularité)
- Gestion de version avancée (par ex. : branches…)
- Les différents savoirs de référence pourront être approfondis

Patrons de conception | Gestion de version | Documentation

Cursus — S3 tous parcours

Heures totales (49h)	**19h** TD et **30h** TP
programme national	12h TD et 20h TP
adaptation locale SAÉ	3h TD et 6h TP
adaptation locale non fléchée	4h TD et 4h TP
Exemples de contribution aux SAÉ	
S3.A.01 Développement appli	3h TD et 6h TP
S3.D.01 Gestion de proj et dév.	3h TD et 6h TP

Coefficients de pondération

UE	*Parcours*	Coeff.
UE 3.1	*parcours A*	15%
	parcours B	15%
	parcours C	15%
	parcours D	15%
UE 3.5	*parcours A*	8%
	parcours B	8%
	parcours C	8%
	parcours D	8%
UE 3.6	*parcours A*	5%
	parcours B	5%
	parcours C	5%
	parcours D	5%

Compétence 1
Partir des exigences et aller jusqu'à une application complète
AC 3 Adopter de bonnes pratiques
AC 4 Vérifier la qualité par les tests

Compétence 6
Situer son rôle et ses missions au sein d'une équipe informatique
AC 2 Intégrer une équipe informatique

Compétence 5
Appliquer une démarche de suivi de projet en fonction des besoins métiers des clients et des utilisateurs
AC 4 Mettre en œuvre un suivi de projet

CC BY

Modalités de contrôle des connaissances

Évaluations prévues :

- Contrôle sur table le lun[illegible]21 septemb[illegible] – 35%
- TP noté le mardi 6 octobre – [illegible]%
- TP noté le mardi 10 novem[illegible] – 1[illegible]%
- Contrôle sur table le m[illegible]di 17 novembre[illegible] 45%

Note : Les dates et mod[illegible]tés définitives seront confirmées pa[illegible]ie standard.

CC BY

Contexte de travail

- Paradigme
  - Programmation orientée objet (POO)
- Langages
  - Diagramme de classe UML
  - Java
- Environnement
  - Papier / crayon
  - Eclipse (sans assistant IA !)

CC BY

Développement de qualité

Ça fonctionne... mais à quel prix ?



vs.



Quel meuble préférez-vous utiliser, entretenir, réparer, etc. ?

CC BY

Développement de qualité

Comment bien utiliser les concepts de la POO pour développer une application ?

Beaucoup d'aspects à considérer :

- Décomposer une application en briques simples
- Factoriser pour éviter la duplication
- Soigner les relations entre les entités
- Préparer les évolutions futures
- Favoriser la réutilisation

Comment distinguer une bonne et une mauvaise conception ?

CC BY

Mauvaise conception

Ce qu'on veut éviter :

- **Rigidité**
  Changer une classe oblige à modifier d'autres classes en cascade
- **Fragilité**
  Changer un aspect entraîne une rupture inattendue dans le système
- **Immobilité**
  Impossible de réutiliser une classe sans traîner toute l'application.

Mais aussi : incertitude et imprévisibilité, difficulté de reprise par une nouvelle équipe, etc.

CC BY

Conception de qualité

Ce qu'on veut avoir :

- **Flexibilité**
  Classes interchangeables, changement localisés
- **Fiabilité**
  Les modifications n'impactent pas le reste du système
- **Réutilisabilité**
  Indépendance des classes, modules, etc.
- etc.

CC BY

Ce qu'on va voir ensemble

1. Principes SOLID

Cinq principes généraux :

- Single responsibility principle
- Open/closed principle
- Liskov's substitution principle
- Interface segregation principle
- Dependency inversion principle

Idée générale :

- Forte **cohésion** = les tâches réalisées par un seul module sont fonctionnellement reliées ;
- Faible **couplage** = les modules sont peu dépendants les uns des autres.

CC BY

Ce qu'on va voir ensemble

2. Documentation

- Documentation technique
- Documentation utilisateur

→ Outils de génération automatique de documentation (Javadoc)

Juste ce qu'il faut :

- Pas de doc = développeur / utilisateur perdu
- Trop de doc = illisible → jamais lu
- Juste ce qu'il faut !

CC BY

Ce qu'on va voir ensemble

3. Patrons de conception (design patterns)

Patron = modèle à suivre / motif à reproduire

Design pattern :

- Modèle de solution à un problème de conception récurrent (organisation et interactions de classes et d'interfaces)
- Solution générale à adapter à chaque situation concrète.

Les design patterns permettent de :

- Bénéficier des bonnes pratiques par des solutions éprouvées
- Respecter les principes de bonne conception
- Réduire les risques durant le développement
- Améliorer la visibilité sur l'architecture $\rightarrow$ facilite la maintenance, l'évolutivité et la réutilisation.

CC BY

Objectif de la ressource

Apprendre à faire abstraction du codage et à raisonner au niveau conceptuel.

Développement de qualité

- Appliquer les design patterns si besoin ;
- Respecter les principes SOLID autant que possible ;
- Veiller à la qualité du code (lisibilité, respect des conventions, documentation, etc.).

Le code finit toujours par être lu plus souvent qu'il n'est écrit
$\rightarrow$ autant le concevoir pour durer.

CC BY

Programmation Orientée Objet

Paradigme : penser en termes d'objets

- Un objet = une entité du monde réel
- Chaque objet a :
  - une structure (attributs)
  - un comportement (méthodes)
- Les objets interagissent entre eux
- La modélisation est essentielle : transcrire le réel en code

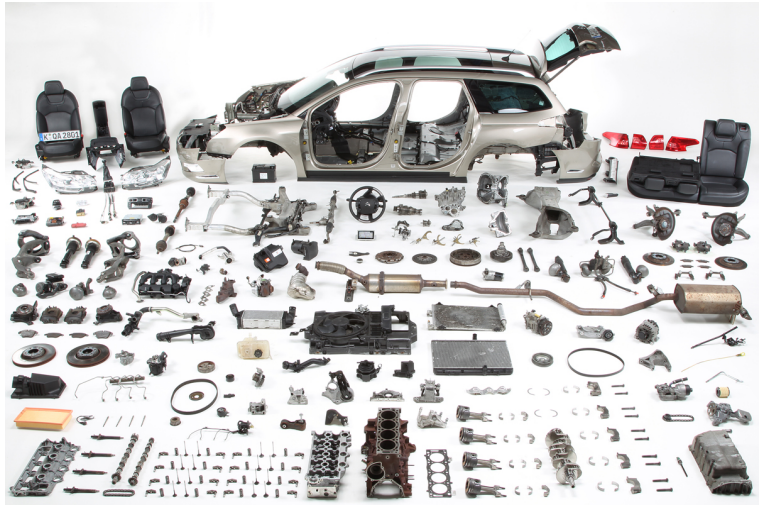


Image : caravenue.com

CC BY

Concepts de base en POO

Construction

- Classe
  = plan de construction
- Objet
  = instance de la classe
- Package
  = regroupement logique

Spécialisation

- Héritage
  = réutilisation + extension
- Classe abstraite
  = modèle partiel
- Interface
  = contrat

CC BY

UML : un langage pour modéliser

Unified Modeling Language

- Langage de modélisation graphique → normalisé et universel
- 14 types de diagrammes pour décrire :
  - la structure,
  - le comportement,
  - les interactions d'un système.

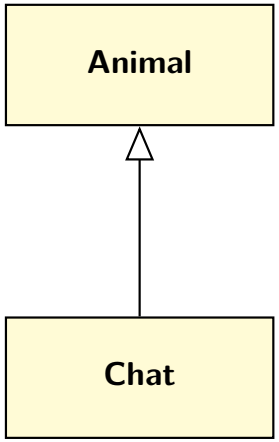
Dans cette ressource : diagrammes de classes

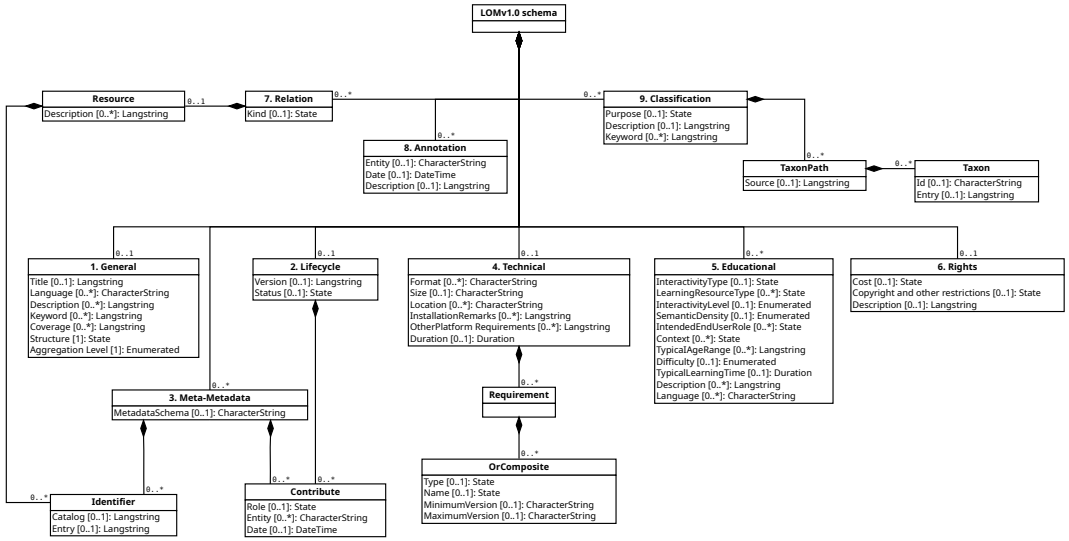
- statiques,
- relations entre classes et interfaces (héritage, dépendances, etc.).

On évoquera aussi les diagrammes de séquence plus tard
(dynamiques, interactions entre objets)

CC BY

Exemple de diagramme de classes





CC BY

UML dans la démarche de conception

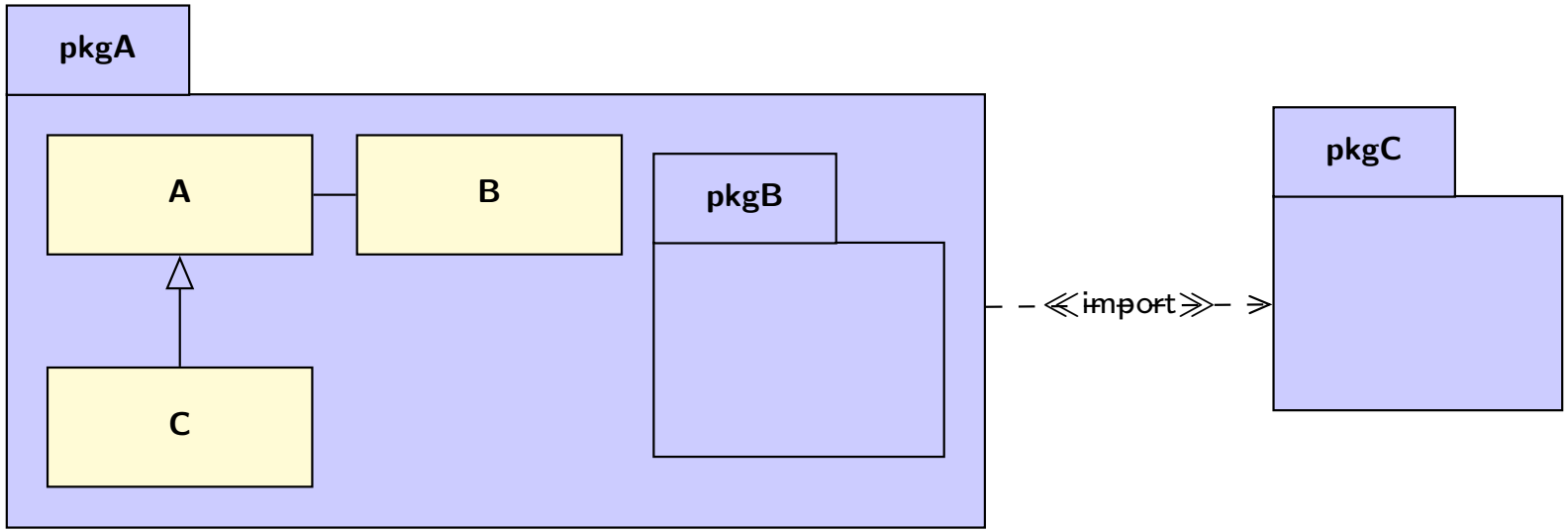
Analyse → Modélisation UML → Codage → Tests → Maintenance.

- **Avant le code** : modéliser les concepts et leurs relations
- **Pendant le développement** : documenter l'architecture
- **Après le code** : faciliter la maintenance et l'évolution

CC BY

Paquetages

- Partitionnement du modèle par regroupement des éléments.
- Contient un ensemble de diagrammes et/ou de paquetages
- Chaque élément possède un nom unique dans ce paquetage.
- Possibilité de définir des relations entre paquetages (cf. dépendances)



CC BY

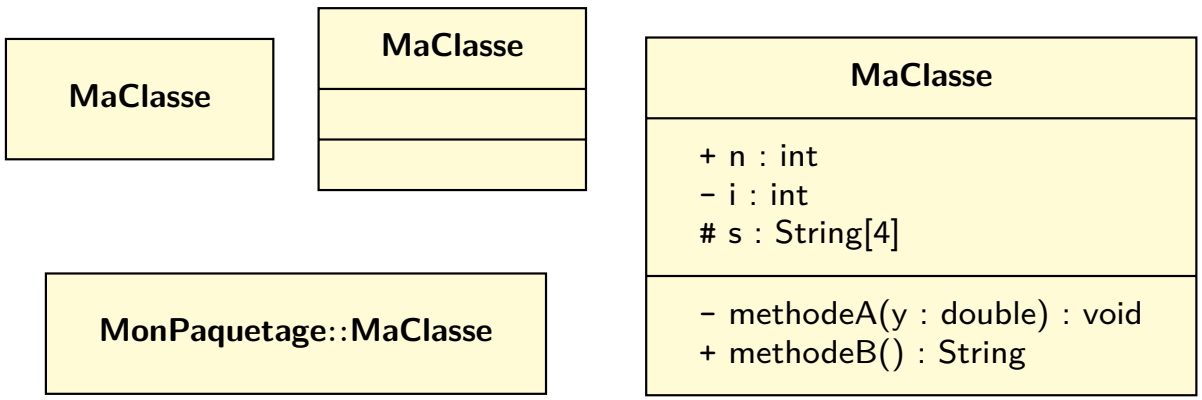
Classes

- Description d'un ensemble d'objets partageant les mêmes attributs, opérations, méthodes, relations et sémantique.
- Selon l'objectif du diagramme, on la décrit par :
  - Nom de la classe

    `<nom>` ou `<paquetage>::<nom>`
  - Attributs

    `<visibilité> <nom> : <type>`
  - Méthodes

    `<visibilité> <nom>(<arg1> : <type1>, ...) : <type retour>`



+ = public

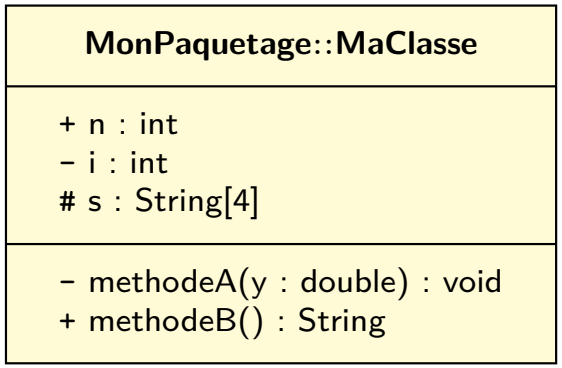
– = private

= protected

∼ = shared

CC BY

Classes → Java




```java
package MonPaquetage;

class MaClasse {

  public int n;
  private int i;
  protected String[4] s;

  /* Constructeurs, etc. */

  private void methodeA(y : double) {
    /* code */
  }

  public String methodeB() {
    /* code */
  }

}
```

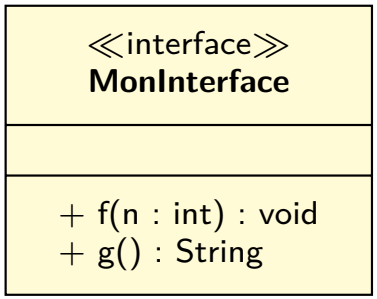
CC BY

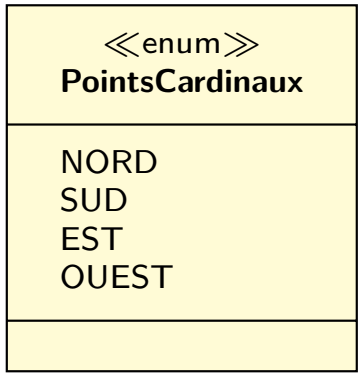
Stéréotypes (et apparentés)

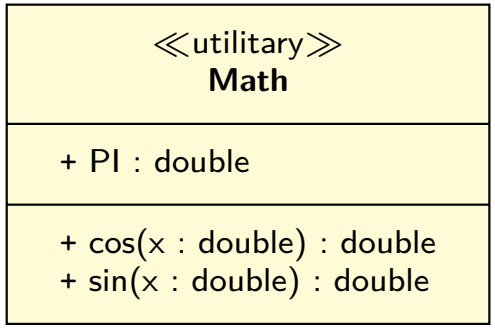
Un stéréotype (noté `<<stéréotype>>`) permet de donner une signification sémantique différente.

On s'en sert, en particulier, pour :

- Les interfaces
- Les énumérations
- Les utilitaires



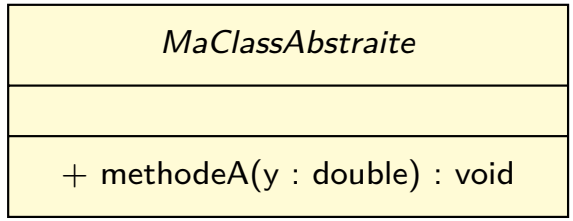


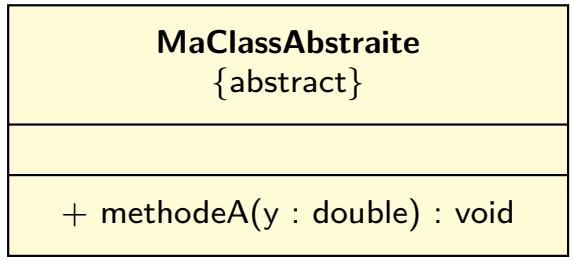


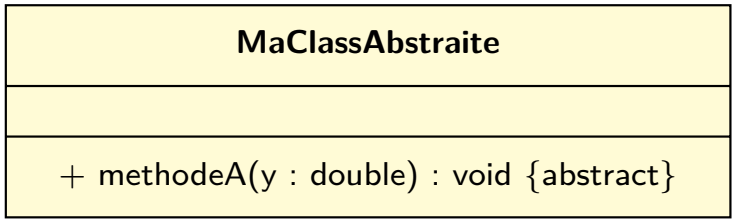
Classes abstraites

Plusieurs notations :

- Nom *en italique*
- Ajout de l'étiquette `{abstract}` sous le nom
- Au moins une méthode est étiquetée abstraite







CC BY

Classes génériques (paramétrées)

Les arguments de généricité sont indiqués en haut à droite.

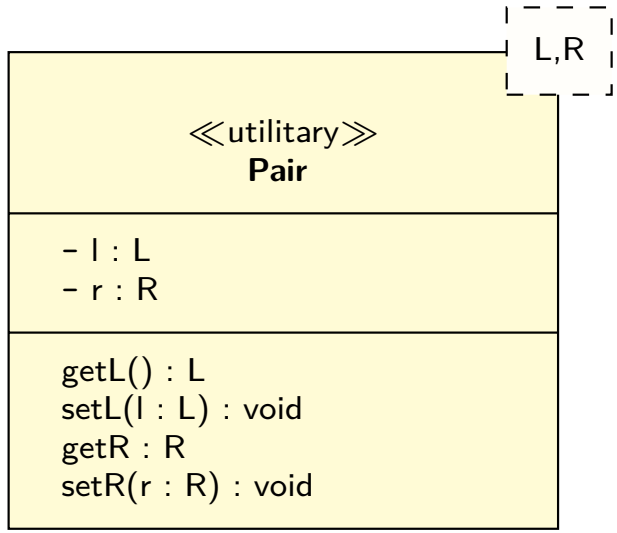
```java
public class Pair<L,R> {

    private L l;
    private R r;

    public Pair(L l, R r) {
        this.l = l;
        this.r = r;
    }

    public L getL() { return l; }
    public R getR() { return r; }

    public void setL(L l) { this.l = l; }
    public void setR(R r) { this.r = r; }
}
```




```java
public static void main(String[] args) {

    Pair<String,Integer> pair = new Pair<>("Test", 1234);

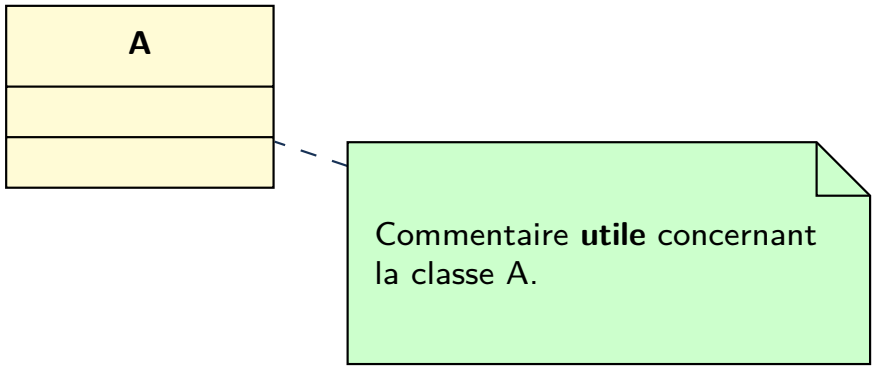
    System.out.println(pair.getL());  // Affiche "Test"
    System.out.println(pair.getR());  // Affiche "1234"

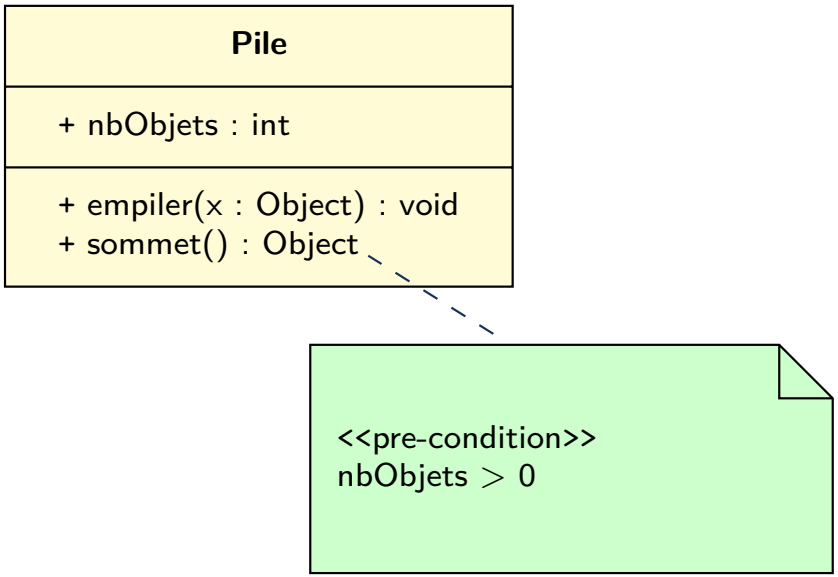
}
```

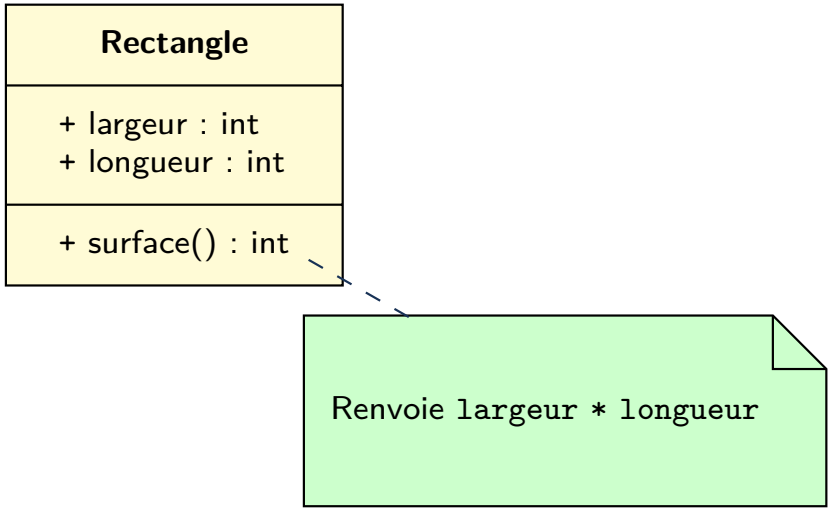
CC BY

Commentaires

- Représentés avec le coin corné
- Permet d'indiquer :
  - Des commentaires,
  - Des pré-post conditions,
  - Des attributs dérivés,
  - etc.





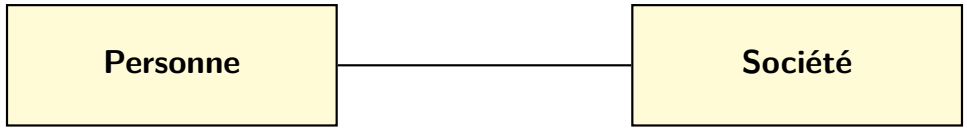


CC BY

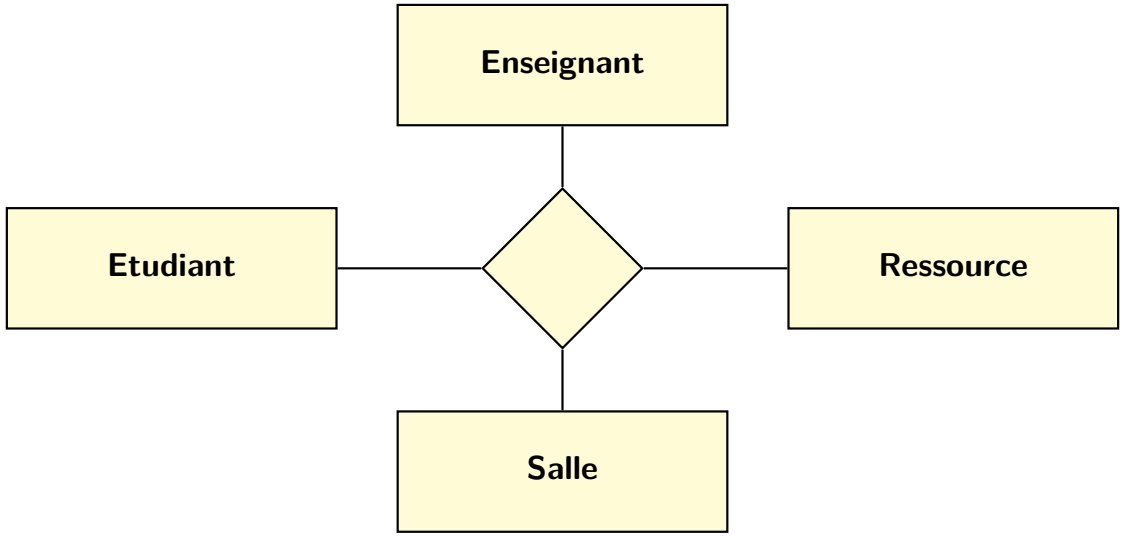
Association

- **Définition :** relation entre plusieurs classes qui entraînent des connexions entre leurs instances.
- **Représentation UML :** trait plein reliant les classes en relation

Relation binaire :



Relation n-aire $(n > 2)$:



CC BY

Nommage

- Les associations peuvent être nommées i.e. identifiées par un texte unique décrivant la sémantique de l'association.
  Utiliser une forme verbale ou passive



- On peut lever l'ambiguïté en indiquant le sens de lecture avec ▶ ou ◀.



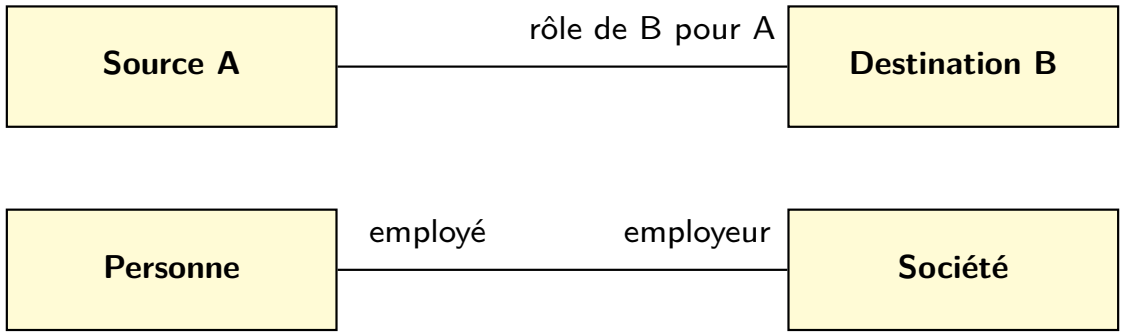
ou



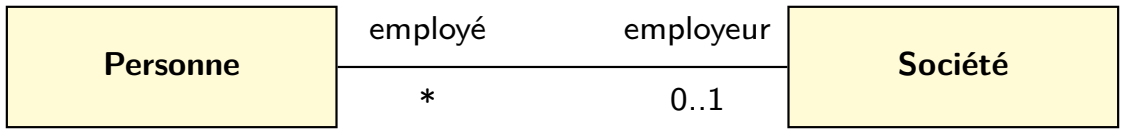
CC BY

Rôles et multiplicité

- Les extrémités peuvent être qualifiées par des rôles indiquant comment une classe source voit une classe destination.
- Le rôle est un pseudo-attribut de la classe source.



- La multiplicité précise le nombre d'instances pouvant être liées par une association.

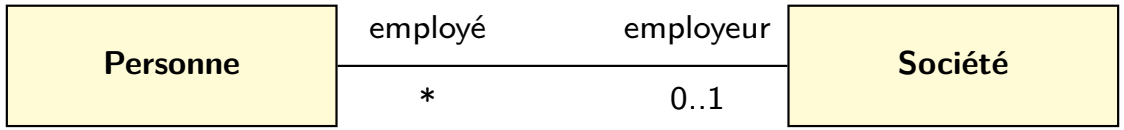


CC BY

Multiplicité

Multiplicités :

UML	Sémantique
1	un et un seul (par défaut)
0..1	zéro ou un
N	exactement N
N..M	entre M et N (inclus)
* ou 0..*	zéro ou plus
1..*	un ou plus




```java
class Personne {

  public Societe employeur;

  /* ... */

}
```

```java
class Societe {

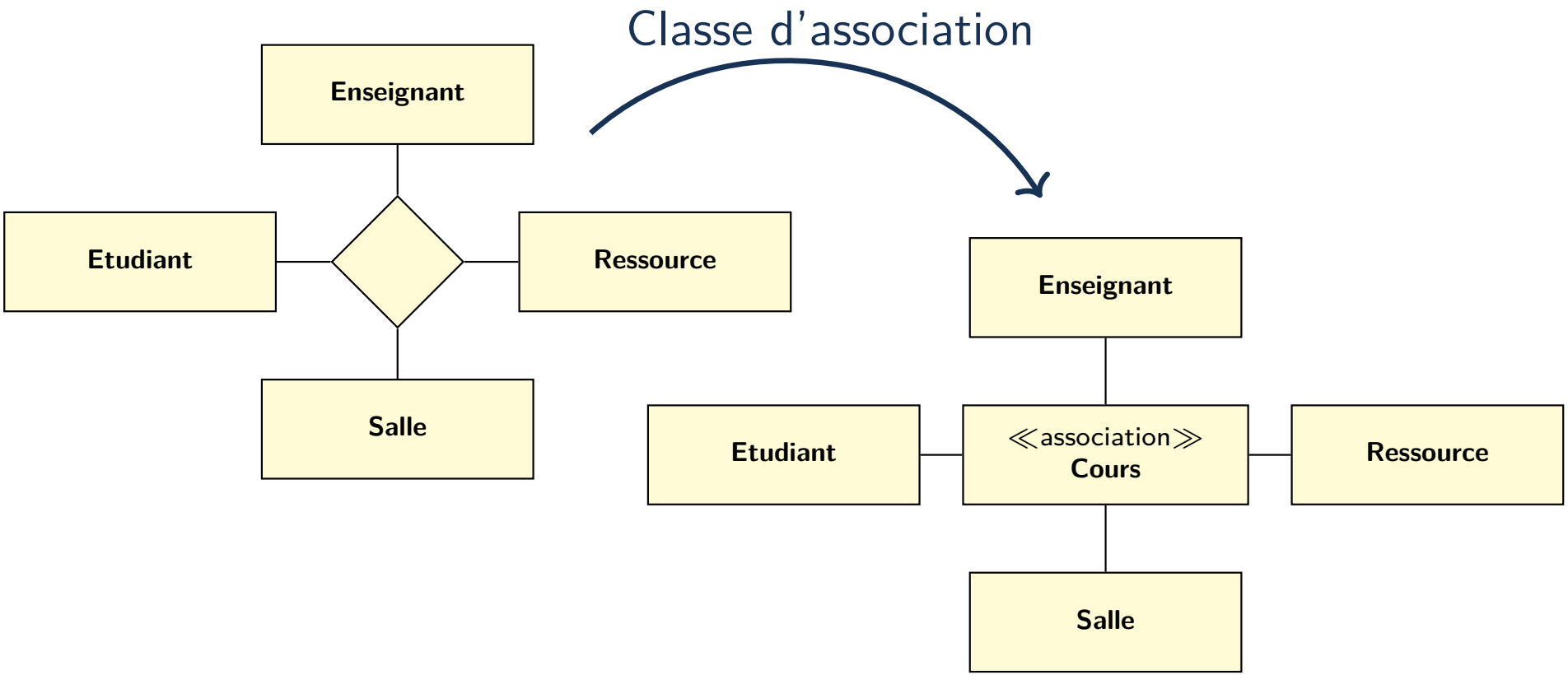
  public Personne[] employes;

  /* ... */

}
```

CC BY

Associations n-aires




```java
class Cours {

  public Etudiant[] etudiants;
  public Enseignant ensseignant;
  public Ressource ressource;
  public Salle salle;

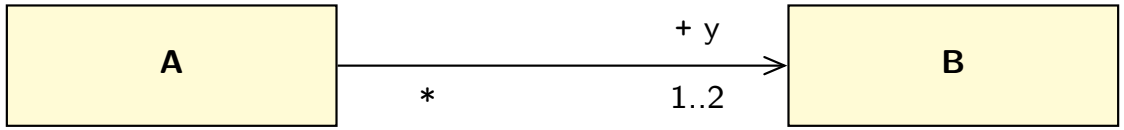
  /* ... */
}
```

```java
class Etudiant {
  public Cours[] cours;
  /* ... */
}
```

```java
class Enseignant {
  public cours[] cours;
  /* ... */
}
```

Association orientée

- Indique l'accessibilité à sens unique
  - À partir d'une instance de A, je peux accéder à son/ses instance(s) de B
  - À partir d'une instance de B, je ne peux pas accéder à son/ses instance(s) de A




```java
class A {

  public B[] y;

  /* ... */

}
```

```java
class B {

  public A[] x;

  /* ... */

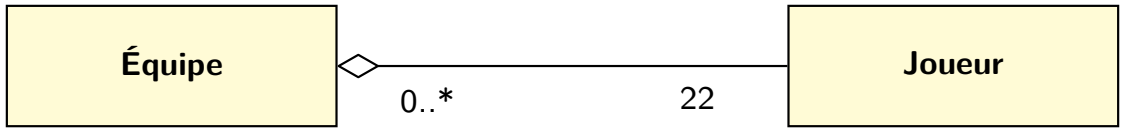
}
```

CC BY

Agrégation et composition

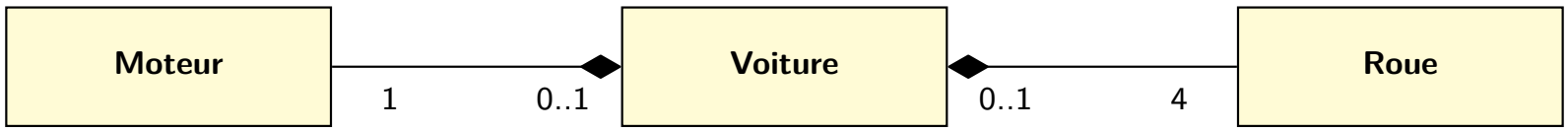
Agrégation :

- Association exprimant une relation « partie–tout »



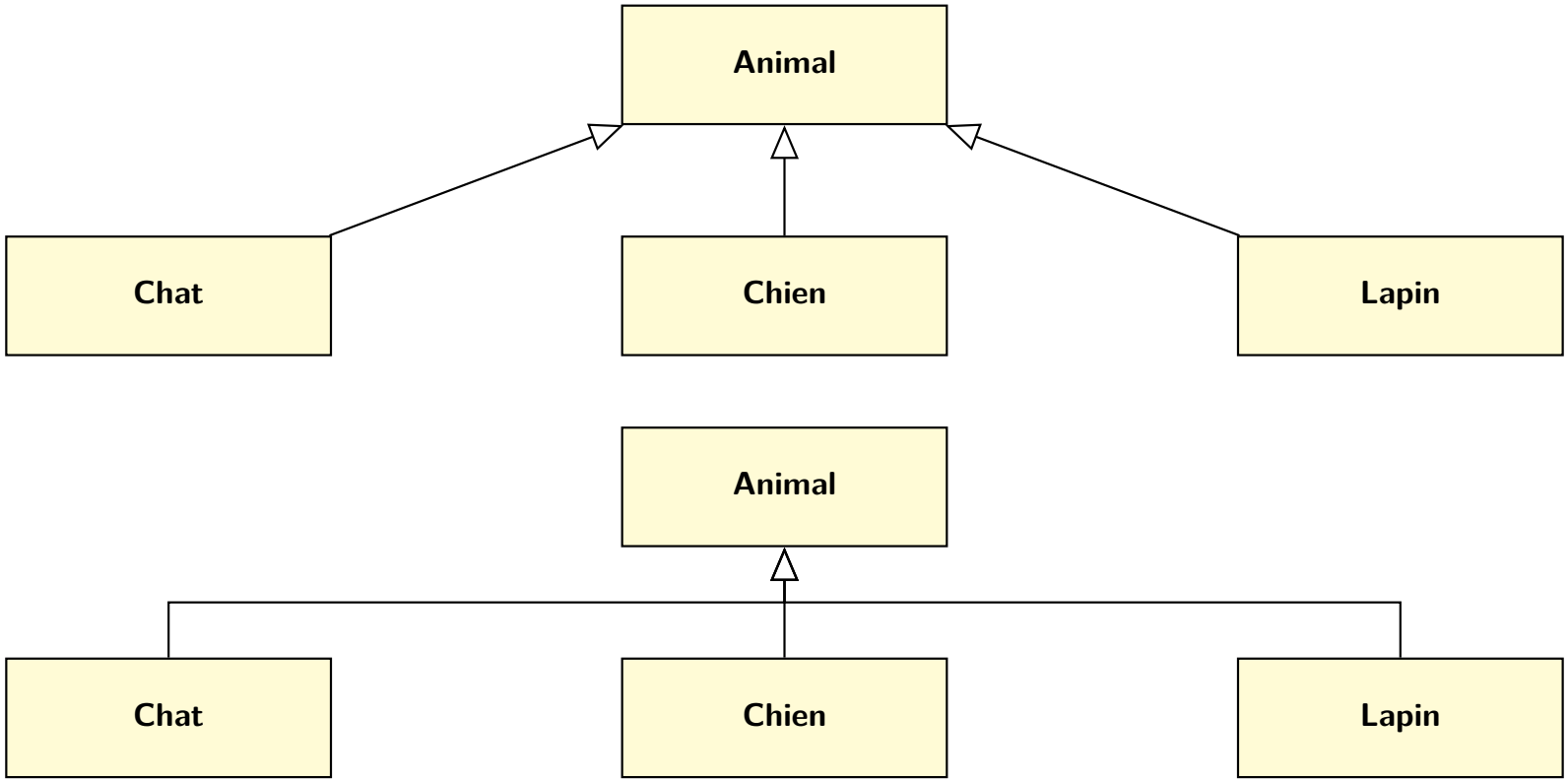
Composition :

- Forme spéciale d'agrégation
- La durée de vie des composants est la même que celle du composite
- La multiplicité du composite est 0..1 ou 1.



Généralisation

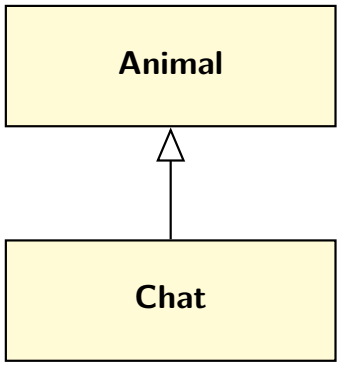
- **Définition :** Relation entre une classe plus générale et une classe plus spécifique.
- **Représentation UML :** Flèche en trait plein avec tête triangulaire



CC BY

Généralisation → Java

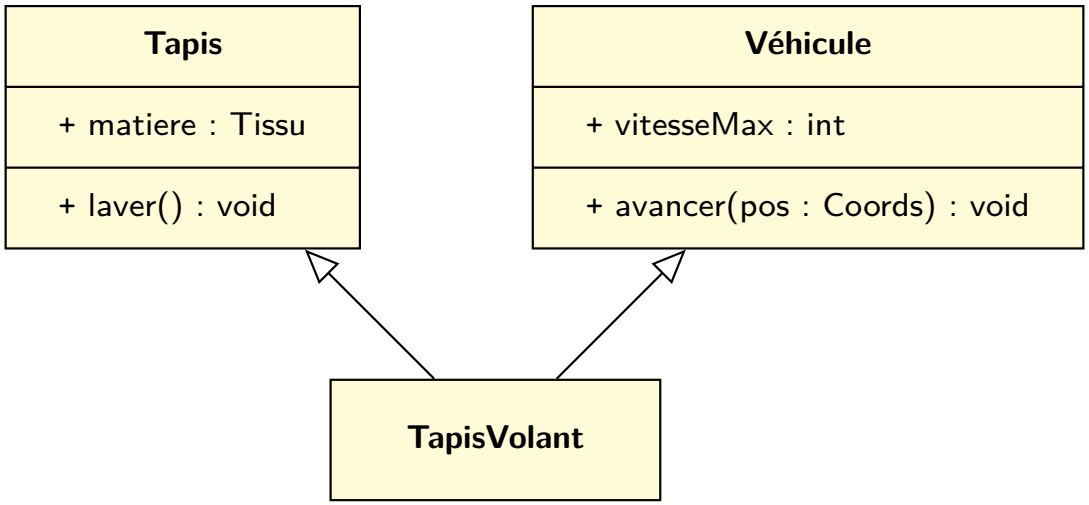
Généralisation = héritage :




```java
class Animal {
  /* ... */
}
```

```java
class Chat extends Animal {
  /* ... */
}
```

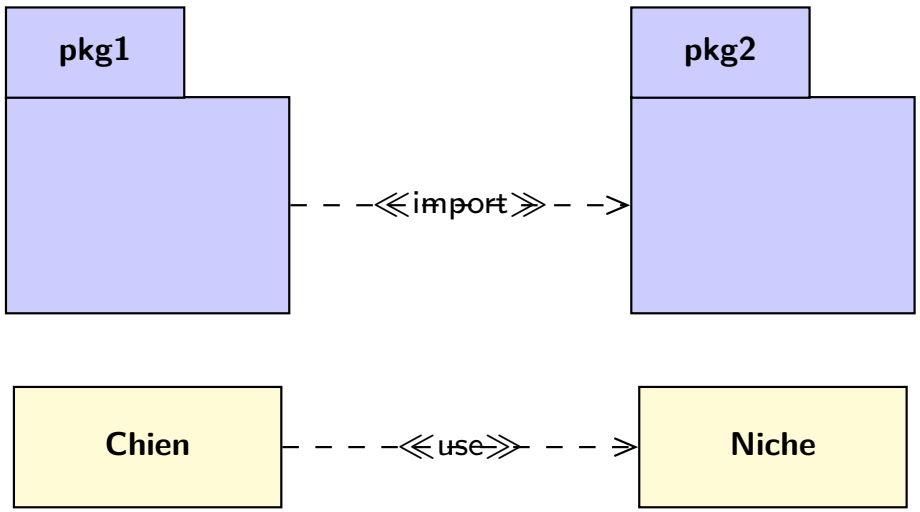
Remarque : UML permet l'héritage multiple... mais pas Java !

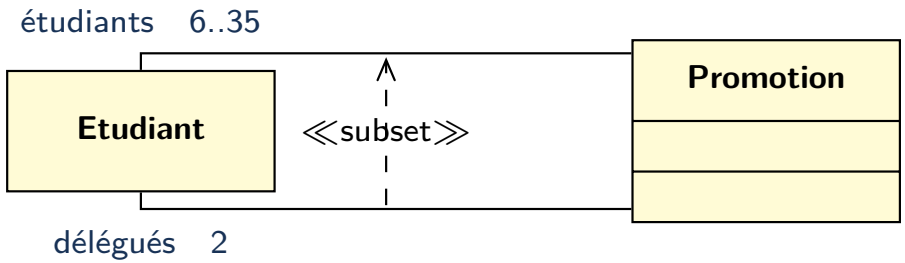


CC BY

Dépendances

- **Définition :** relation **non structurelle** unidirectionnelle indiquant qu'un changement dans la cible provoque un changement dans la source.
- **Représentation UML :** flèche pointillée éventuellement stéréotypée entre deux éléments.

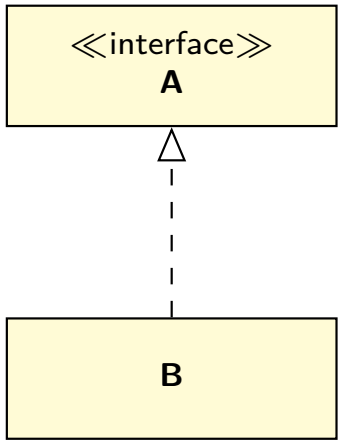


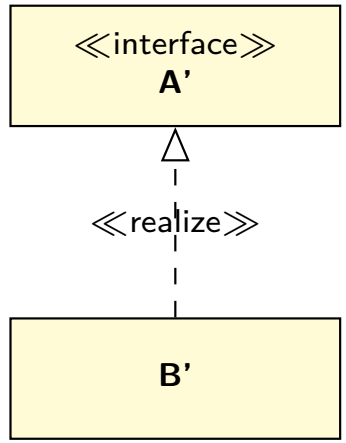


CC BY

Réalisation d'interface

- **Représentation UML :** flèche pointillée avec tête en triangle (éventuellement le stéréotype `<<realize>>`)






```java
interface A {
  /* ... */
}
```

```java
class B implements A {
  /* ... */
}
```

CC BY